



Univerzitet u Tuzli
Fakultet elektrotehnike
Telekomunikacije i informacione tehnologije

Razvoj telekomunikacijske programske podrške

Akadska godina 2025/26

ZADACA BR. 2

Razvoj REST API aplikacije

Tuzla, 2025.

Cilj ove zadaće je da studenti kroz praktičan timski rad razviju potpunu **REST API aplikaciju** koristeći Python ekosistem (FastAPI, SQLAlchemy, SQLite). Studenti rade u **parovima** – zajedno biraju domenu aplikacije, definišu model podataka i implementiraju kompletnu serversku logiku uz obaveznu kolaboraciju putem Git-a.

1. Formiranje parova i odabir domene

Svaki par treba odabrati jednu **domenu aplikacije** (npr. bibliotekarski sistem, evidencija opreme, rezervacija sala, katalog filmova, upravljanje projektima i sl.) i definisati odgovarajući model podataka koji će biti osnova cijele aplikacije.

Studenti trebaju koristiti Piazza grupu za:

- dogovor oko odabira domene i podjele zadataka,
- razmjenu ideja i pitanja,
- koordinaciju rada na projektu.

2. Model podataka

Na osnovu odabrane domene, **svaki student definira svoj SQLAlchemy entitet** koji će biti pohranjen u zajedničku SQLite bazu podataka. Entiteti trebaju biti tematski povezani (npr. knjiga i autor, soba i rezervacija).

Svaki entitet mora imati:

- automatski generirani primarni ključ (`id`),
- najmanje **5 polja** različitih tipova (`string`, `int`, `float`, `bool`, `Optional` i sl.),
- logički smisljena polja u kontekstu odabrane domene.

Pored glavnog entiteta, potrebno je definisati i **Pydantic / SQLAlchemy sheme** za:

- kreiranje novog resursa (`XCreate`),
- djelimično ažuriranje resursa (`XUpdate` – sva polja opcionalna).

Primjer (studentski registar):

```
class Student(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    ime: str
    prezime: str
    godina: int
    indeks: str
    email: Optional[str] = None
```

3. REST API endpointi

Aplikacija mora implementirati **kompletni CRUD** nad odabranim entitetom kroz sljedeće HTTP endpointe:

Metoda	Ruta	Opis
GET	/resursi	Dohvatanje liste svih resursa (sa opcionalnim query filterom)
GET	/resursi/{id}	Dohvatanje jednog resursa po ID-u
POST	/resursi	Kreiranje novog resursa (status 201)
PUT	/resursi/{id}	Potpuna zamjena resursa
PATCH	/resursi/{id}	Djelimično ažuriranje resursa
DELETE	/resursi/{id}	Brisanje resursa (status 204)

Dodatni zahtjevi za endpointe:

- GET /resursi mora podržavati **najmanje jedan query parametar** za filtriranje (npr. po godini, kategoriji, statusu i sl.).
- Svaki endpoint koji prima id mora vraćati **HTTP 404** ukoliko resurs nije pronađen, uz odgovarajuću poruku.
- PATCH endpoint mora koristiti `exclude_unset=True` kako bi se ažurirala samo polja koja su eksplicitno poslana u zahtjevu.
- Aplikacija mora koristiti **dependency injection** (Depends) za upravljanje sesijom baze podataka.

4. Podjela rada i Git kolaboracija

Zadatak se radi uz obavezno korištenje Git workflow-a. Zadatak je postavljen kao **Group Assignment na GitHub Classroom-u**. Svaki par studenata ima jedan zajednički repozitorij.

Podjela implementacije

Svaki student implementira **kompletni CRUD** za jedan zasebni resurs unutar iste aplikacije. Na taj način oba studenta prolaze kroz cijeli razvojni proces: definisanje modela, pisanje shema i implementaciju svih HTTP metoda.

Par zajedno bira domenu aplikacije i dogovara se koja dva međusobno povezana resursa će implementirati. Na primjer:

Domena	Student A	Student B
Biblioteka	/knjige	/autori
Teretana	/clanovi	/treninzi
Kino	/filmovi	/projekcije
Bolnica	/pacijenti	/ljekari
Hotel	/sobe	/rezervacije

Svaki student za svoj resurs implementira:

- model podataka i sheme (XCreate, XUpdate),
- GET /resursi – lista svih resursa s najmanje jednim query filterom,
- GET /resursi/{id} – dohvaćanje po ID-u (HTTP 404 ako ne postoji),
- POST /resursi – kreiranje novog resursa (status 201),
- PUT /resursi/{id} – potpuna zamjena resursa,
- PATCH /resursi/{id} – djelimično ažuriranje (exclude_unset=True),
- DELETE /resursi/{id} – brisanje resursa (status 204).

Zajednički dijelovi projekta (database.py, main.py) implementiraju se kroz Git workflow: jedan student kreira fajl, drugi ga proširuje kroz merge.

Pravila rada

- Oba studenta rade u posebnim granama (*branches*).
- Svaka funkcionalnost implementira se u posebnoj grani.
- Sve izmjene moraju biti *commit*-ovane s jasnim i opisnim porukama.
- Sve grane moraju biti *push*-ovane na GitHub.
- Prilikom spajanja grana u **main** **obavezno** je koristiti:

```
git merge --no-ff
```
- Fast-forward merge **nije** **dozvoljen**.
- Preporučuje se da drugi član tima pregleda izmjene prije merge-a (pull request).

Minimalni Git zahtjevi (po studentu)

- najmanje **3 commita**,
- najmanje **1 vlastita grana**,

- najmanje **1 merge** svojih izmjena u zajednički projekat.

Git historija bit će dio ocjenjivanja – commit poruke moraju biti smislene i opisne (izbjegavati generičke poruke poput *"update"* ili *"fix stuff"*).

5. Korištenje AI alata

Studenti mogu koristiti AI alate (npr. GitHub Copilot, ChatGPT i sl.) kao pomoć pri pisanju koda. Međutim, studenti su odgovorni da:

- razumiju generisani kod,
- provjere njegovu ispravnost,
- po potrebi ga prilagode ili isprave.

U `README.md` datoteci potrebno je navesti kratku evidenciju korištenja AI alata koja uključuje:

- naziv alata (npr. GitHub Copilot, ChatGPT),
- model koji je korišten (ako je poznat, npr. GPT-4, Copilot model),
- najmanje **2 primjera** promptova koje ste koristili,
- kratko objašnjenje kako vam je AI pomogao i da li ste morali prilagoditi generisani kod.

6. Struktura repozitorija

Repozitorij treba imati sljedeću strukturu:

```
project/  
|-- main.py           # pokretanje FastAPI aplikacije i  
    startup event  
|-- database.py       # konfiguracija enginea i get_session  
    dependency  
|-- models_a.py       # SQLAlchemy entitet i sheme -- Student  
    A  
|-- models_b.py       # SQLAlchemy entitet i sheme -- Student  
    B  
|-- routes_a.py       # API endpointi za resurs A  
|-- routes_b.py       # API endpointi za resurs B  
|-- README.md
```

Opis datoteka:

`database.py` – Sadrži konfiguraciju SQLite baze podataka, kreiranje `engine-a`, funkciju `create_db_and_tables()` i generator `get_session()` koji se koristi kao dependency. Ovu datoteku pišu oba studenta zajedno (ili jedan kreira, drugi dopunjuje kroz merge).

`models_a.py` i `models_b.py` – Svaki student definira svoj SQLAlchemy entitet (table model) te odgovarajuće sheme `XCreate` i `XUpdate`.

`routes_a.py` i `routes_b.py` – Svaki student implementira sve API rute za svoj resurs, registrirane na zasebnom `APIRouter` objektu.

`main.py` – Kreira FastAPI instancu, uključuje oba routera i definira `startup` event koji poziva `create_db_and_tables()`. Integracija se radi kroz Git workflow: jedan student kreira fajl s prvim routerom, drugi dodaje drugi router kroz merge.

`README.md` – Treba sadržavati:

- kratak opis domene i svrhe aplikacije,
- upute za pokretanje projekta (instalacija zavisnosti, pokretanje servera),
- pregled svih endpointa s primjerima zahtjeva,
- kratku evidenciju korištenja AI alata.

7. Napomene

- Nazivi funkcija, varijabli i ruta trebaju biti na **engleskom jeziku**, dok komentari u kodu trebaju biti napisani na **bosanskom jeziku**.
- Aplikacija se mora uspješno pokrenuti komandom:

```
uvicorn main:app --reload
```

- Svi endpointi moraju biti dostupni i testirani putem automatski generisane Swagger dokumentacije na adresi `http://localhost:8000/docs`.
- Tokom odbrane zadaće od studenata će biti traženo da kratko objasne dijelove svog rješenja. Važno je da razumijete kod koji ste implementirali.
- **Nije dozvoljeno** koristiti istu domenu aplikacije kao primjer koji je prikazan na predavanjima (studentski registar).